Quality Assurance Testing Documentation

Project Title: HireFlow – Decision Engine for Recruitment Workflows

Domain: Domain 1 – AI Systems & Agentic Workflow Automation

Group Name: Fantastic Four

Team member 1	Tsang Da Xin
Team member 2	Patrick Woo Soon Tat
Team member 3	Tan E Jane
Team member 4	Nareen Raj

Contents

Document Control....3

Preliminary Round (Test Strategy & Planning)....4

1.0 Scope & Requirements Traceability....4

1.1 In-Scope Core Features....4

1.2 Out-of-Scope....5

2.0 Risk Assessment & Mitigation Strategy....6

3.0 Test Environment & Execution Strategy....9

3.1 Unit Testing....9

3.2 Integration Testing....9

3.3 Test Environment (CI/CD Practice)....10

3.4 Regression Testing & Pass/Fail Rules....10

3.5 Test Data Strategy....11

3.6 Passing Rate Threshold....11

4.0 CI/CD Release Thresholds & Automation Gates....12

4.1 Integration Thresholds (Merging to Main)....12

4.2 Deployment Gate....12

5.0 Test Case Specifications....13

6.0 AI Output & Boundary Testing....16

6.1 Prompt / Response Test Pairs....16

6.2 Oversized / Larger Input Test....17

6.3 Adversarial / Edge Prompt Test....18

6.4 Hallucination Handling....18

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 – HireFlow
Team Repo URL	https://github.com/Patr1xk/UM_HACK_2026
Project Board URL	https://www.notion.so/UM-HACKATHON-2026-3454979b379380f383cac02aba7ce924?source=copy_link

The primary objective of this Quality Assurance Testing Documentation is to ensure that HireFlow can reliably execute the core early-stage hiring workflow, including resume submission, structured profile extraction, candidate matching, decision routing, interview scheduling trigger, and notification delivery.

The testing strategy is designed to verify that the system behaves correctly under both normal and abnormal conditions, with particular focus on GLM-assisted reasoning quality, workflow state consistency, input validation, and external API reliability. In addition to functional correctness, this document defines the planned approach for risk-based testing, release thresholds, and boundary testing to ensure the MVP remains stable, explainable, and technically credible throughout UMHackathon 2026.

Preliminary Round (Test Strategy & Planning)

1.0 Scope & Requirements Traceability

This section defines the testing boundaries for HireFlow and ensures that all planned tests are directly aligned to the system's in-scope functional requirements. By mapping test activities to core user requirements, the team can verify that the MVP is tested systematically, while avoiding unplanned feature expansion beyond the intended scope.

1.1 In-Scope Core Features

The following core features are included in the testing scope for the HireFlow MVP:

- **Resume Submission & Profile Extraction**

  The system accepts resume input from a candidate or recruiter and uses Z AI GLM to generate a structured candidate profile from unstructured resume content.

- **Job Criteria Modelling & Candidate Matching**

  The system interprets job requirements into structured criteria and evaluates candidate-job alignment, producing match scores, strengths, gaps, and reasoning outputs.

- **AI-Driven Decision Routing**

  The system routes candidates to the next workflow stage such as shortlist, review, or reject based on structured matching outputs and workflow rules.

- **Interview Scheduling Trigger**

  For shortlisted candidates, the system triggers interview scheduling by interacting with calendar integration and updating workflow state accordingly.

- **Notification & Status Updates**

  The system sends relevant workflow updates, decisions, and scheduling notifications to recruiters and candidates through supported communication channels.

1.2 Out-of-Scope

The following features are intentionally excluded from the preliminary round testing scope:

- Full onboarding lifecycle after hiring confirmation
- Real-time recruiter–candidate conversational chat
- Advanced analytics dashboards and long-term hiring performance reporting
- Full ATS migration and enterprise HR system integration
- Candidate profile editing and account personalization features
- UI theming and non-essential interface customization
- Post-hire employee lifecycle management

2.0 Risk Assessment & Mitigation Strategy

Quality Assurance risks for HireFlow are identified based on the system's architecture, GLM dependency, and workflow execution logic. Each risk is evaluated using a **5×5 risk assessment matrix**, where: **Risk Score = Likelihood × Severity**

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Invalid or malformed GLM output (non-JSON / incomplete fields)	4	5	20 (Critical)	Enforce strict JSON schema validation, retry mechanism, and fallback handling before passing output downstream	Inject malformed prompts / simulate invalid responses and verify parser rejection + retry logic
Incorrect candidate-job matching due to weak reasoning or missing context	3	4	12 (High)	Improve prompt structure, enforce structured inputs (profile + job criteria), and validate key fields before scoring	Test with edge cases (incomplete resume, conflicting skills) and compare expected vs actual match outputs
Wrong decision routing (e.g., strong candidate rejected)	3	5	15 (High)	Introduce decision thresholds, confidence scoring, and allow human	Simulate borderline candidates and verify routing logic (shortlist / review /

				review override for critical cases	reject) correctness
Workflow state inconsistency (stage not updated after decision)	3	5	15 (High)	Use persistent state storage with atomic updates and validation checks before transition	Force interruption between stages and verify system maintains consistent workflow state
Scheduling failure due to external Calendar API issues	4	3	12 (High)	Implement retry logic, fallback queue, and allow manual scheduling if API fails	Simulate API failure or timeout and verify retry and fallback behaviour
Notification delivery failure (email not sent)	3	3	9 (Medium)	Implement retry mechanism and delivery status tracking	Trigger notification failures and verify system logs failure and retries correctly
Oversized input (resume exceeds token limit)	4	4	16 (Critical)	Apply chunking, filtering, and input validation before sending to GLM	Test with large resumes exceeding limit and verify chunking or rejection behaviour

Hallucinated reasoning (fabricated skills or incorrect justification)	3	4	12 (High)	Constrain prompts, enforce structured outputs, and cross-check with extracted data fields	Provide ambiguous inputs and verify reasoning consistency with actual data
High latency in GLM response affecting user experience	4	3	12 (High)	Limit context size, cache repeated inputs, and restrict deep reasoning steps	Measure response time under repeated calls and ensure it meets acceptable thresholds
External API dependency failure (GLM / email / calendar unavailable)	3	5	15 (High)	Implement graceful degradation, retry strategy, and workflow pause instead of failure	Simulate API downtime and verify system does not crash and preserves workflow state

3.0 Test Environment & Execution Strategy

This section defines where HireFlow will be tested, how tests will be executed across development stages, how test data will be prepared, and the rules that govern pass/fail decisions during the preliminary round.

3.1 Unit Testing

- **Scope**: Unit testing focuses on isolated logic inside core components, including input validation, workflow state transitions, prompt construction, response parsing, schema validation, and decision routing rules.
- **Execution**: Unit tests are written and executed during development and are also triggered automatically through the CI pipeline on every code push or pull request.
- **Isolation**: External dependencies such as the database, GLM API, calendar API, and notification services are mocked so that each unit test evaluates only the internal logic of the component being tested.
- **Pass Condition**: A unit test passes only when the output exactly matches the expected result for happy cases, negative cases, and edge cases. This includes correct validation behaviour, proper fallback handling, and correct return structure for structured outputs.

3.2 Integration Testing

- **Scope:** Integration testing verifies that connected system components work correctly together. For HireFlow, this includes flows such as:
  - resume submission → profile extraction
  - profile extraction → candidate matching
  - matching → decision routing
  - decision routing → scheduling trigger
  - scheduling → notification delivery
- **Execution:** Integration tests are executed after the relevant modules have passed unit testing and are sufficiently complete to interact as one workflow.
- **Workflow:** These tests use real interactions between backend logic, database, parser/validator, and supported API layers where possible, while minimizing unnecessary mocking. The goal is to verify that data moves correctly across components and that workflow state is updated consistently.

- **Pass Condition:** An integration test passes when connected modules operate together correctly, return the expected output, and preserve workflow consistency without breaking downstream stages.

3.3 Test Environment (CI/CD Practice)

- **Local Testing:** Developers perform local testing on localhost during implementation to verify component behaviour, UI interaction, and early debugging of workflow logic.
- **Staging / CI Environment:** GitHub Actions is used to run automated checks such as build validation, unit tests, and selected integration tests on every push and pull request.
- **CI/CD Automated Practice:** The automated pipeline is designed so that branch-level checks validate build stability and basic unit correctness, while pull requests to the main branch trigger stricter checks such as integration tests, linting, and selected regression coverage.

3.4 Regression Testing & Pass/Fail Rules

- **Execution Phase:** Regression testing is performed whenever major workflow logic changes are introduced or when a branch is merged into the main branch. Testing focuses on all in-scope MVP features:
  - profile extraction
  - candidate matching
  - decision routing
  - scheduling trigger
  - notification/status updates
- **Pass/Fail Condition:** A test case is considered passed only when the actual result fully matches the expected result. Any mismatch in output structure, workflow stage, API response handling, or user-facing behaviour is considered a failed case and must be recorded for review.
- **Continuation Rule:** Critical workflow tests must pass before the team proceeds to later-stage testing or demo validation. If critical tests fail, the build should not be treated as demo-ready.

3.5 Test Data Strategy

- **Manual Test Data:** The team will prepare mock candidate resumes, job descriptions, and recruiter actions covering multiple scenarios, including:
  - strong candidate match
  - weak candidate match
  - incomplete resume input
  - oversized resume input
  - shortlist / review / reject routing scenarios
- **Automated Test Data:** Seed data and predefined JSON fixtures will be used for automated tests, including mock candidate profiles, job criteria, workflow states, and simulated GLM responses. This ensures repeatable and consistent testing across environments.

3.6 Passing Rate Threshold

A minimum overall passing rate of 85% is required across all executed tests during the preliminary round. However, for critical workflow tests, the passing threshold is 100%, with no exceptions. These include:

- profile extraction validation
- candidate matching output structure
- decision routing correctness
- workflow state consistency
- fallback handling for invalid GLM responses

If any critical test fails, the build is considered unstable and should not proceed to final demo validation until corrected.

4.0 CI/CD Release Thresholds & Automation Gates

This section defines the **quality gates and measurable thresholds** that must be satisfied before code can be merged into the main branch or deployed for demonstration. If any threshold is not met, the pipeline blocks progression to ensure system stability and reliability.

4.1 Integration Thresholds (Merging to Main)

The main branch is treated as a stable source of truth, therefore strict continuous integration checks are enforced before merging.

Checks	**Requirements**	**Project**	**Pass/Fail**
Automatic Build	Zero build errors	0 errors	Passed
Unit Tests	100% passing rate	100%	Passed
Code Quality (Linting)	Zero critical linting errors	98% compliant	Passed
Test Coverage	Minimum 80% coverage	82%	Passed

4.2 Deployment Gate

Checks	**Requirements**	**Project**	**Pass/Fail**
Regression Test	Minimum 90% pass rate	92%	Passed
AI Output Pass Rate	Minimum 80% of evaluated prompts pass validation criteria	85%	Passed
Critical Bugs (P0/P1)	Zero critical bugs	0	Passed
API Performance	Average response time < 800ms (excluding GLM latency)	520ms	Passed
Security	No exposed API keys or sensitive credentials	Clean	Passed

5.0 Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow): Resume Submission → Matching → Decision Routing → Scheduling**	Verify that a valid candidate resume can be submitted, processed into a structured profile, matched against job criteria, routed to the correct decision stage, and scheduled successfully if shortlisted. This validates that the full MVP workflow operates correctly end to end.	1. Open the HireFlow system. 2. Submit a valid resume for a defined job opening. 3. Trigger profile extraction. 4. Run candidate-job matching. 5. Review generated match score and reasoning. 6. Confirm that the system routes the candidate to "Shortlist." 7. Trigger interview scheduling. 8. Verify notification/status update is sent.	The system should extract a valid structured candidate profile, generate a match result with reasoning, route the candidate to the correct workflow stage, schedule the interview successfully, and return updated status to the recruiter and candidate without workflow inconsistency.	Successfully, the resume was processed, a structured profile was generated, the candidate was matched and shortlisted, scheduling was completed, and status updates were displayed correctly. **Status: Passed**
TC-02	**Specific Case (Negative / Edge): Invalid GLM**	Verify that the system handles malformed or incomplete GLM output	1. Submit a test resume. 2. Simulate or inject an invalid GLM response such as missing	The system should reject the invalid output, prevent workflow	The invalid response was rejected by the parser, the workflow

	Output During Profile Extraction	safely without corrupting the workflow or proceeding to the next stage incorrectly.	required JSON fields or malformed schema. 3. Observe parser and validator behaviour. 4. Check whether workflow progression is blocked. 5. Verify fallback or retry logic is triggered.	progression, trigger retry or fallback handling, log the failure, and avoid writing inconsistent profile data to the database.	did not progress to matching, and fallback handling was triggered correctly. **Status: Passed**
TC-03	**NFR (Performance): Profile Extraction & Matching Response Time**	Verify that the core reasoning workflow remains responsive under normal concurrent usage.	1. Prepare 20 valid resume submissions. 2. Trigger profile extraction and matching requests concurrently. 3. Measure average response time and error rate. 4. Record whether workflow states remain consistent.	Average processing latency for the tested workflow should remain within the acceptable threshold defined for MVP testing, with no critical workflow failures and no corrupted state transitions.	Average response time remained within the acceptable threshold, no critical errors occurred, and workflow states were updated consistently. **Status: Passed**
TC-04	**NFR (Robustness / Load): Scheduling and Notification**	Verify that the system remains stable when external integrations such as calendar or notification	1. Create a shortlisted candidate workflow. 2. Simulate delayed or failed Calendar API response. 3. Trigger scheduling	The system should not crash or corrupt workflow state. Scheduling/notification	The system preserved workflow state, retry logic executed correctly,

	under External API Delay	services respond slowly or temporarily fail.	flow. 4. Observe retry, fallback, and workflow status handling. 5. Repeat for notification delivery failure.	actions should retry or move into fallback handling, and the workflow should remain recoverable.	and the failed external action did not break the candidate workflow. **Status: Passed**

6.0 AI Output & Boundary Testing

6.1 Prompt / Response Test Pairs

The following prompt-response test pairs are designed around the core GLM-supported tasks in HireFlow.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Extract a structured candidate profile from the following resume text for hiring evaluation. Return JSON with fields: name, years_experience, education_level, skills, summary.	Output must be valid JSON, include all required fields, avoid fabricated skills or experience, and accurately reflect the resume content. Missing or malformed fields are considered failure.	GLM returned valid JSON with all required fields. Skills and experience were aligned with the source resume, and no unsupported attributes were introduced.	**Pass**
AI-02	Given the structured candidate profile and job criteria, evaluate candidate-job fit and return JSON with fields: overall_score, strengths, gaps, recommendation, reasoning.	Output must be valid JSON, recommendation must align with score and reasoning, strengths/gaps must be relevant to provided inputs, and no hallucinated criteria should appear.	GLM returned a structured match result with coherent strengths, identified relevant gaps, and produced a recommendation consistent with the profile and job criteria.	**Pass**
AI-03	Based on the match result and current workflow state, recommend the next hiring stage and return JSON with fields:	Output must be valid JSON, decision must be one of the allowed values (shortlist, review, reject), confidence	GLM returned a valid structured decision. Decision type matched allowed workflow states, confidence	**Pass**

	decision_type, decision_reason, confidence_score.	score must be numeric, and reasoning must not contradict the match result.	score was valid, and reasoning was consistent with the matching output.	

6.2 Oversized / Larger Input Test

This test verifies how the system behaves when resume input exceeds the acceptable input limit for the GLM-assisted extraction flow.

Field	**Details**
Maximum Input Size	Approximately 8,000 tokens per reasoning call for MVP testing
Input used while testing	Oversized resume input exceeding the normal limit (e.g. multiple-page resume with repeated project descriptions and appended extra text)
Expected Behaviour	The system should not fail silently. It should either chunk the input into smaller processable segments or reject the request with validation feedback if the input is unusable.
Actual Behaviour	The system applied input segmentation and preserved workflow continuity. Where excessive redundancy was detected, non-essential content was filtered before prompt construction.
Status	**Pass**

6.3 Adversarial / Edge Prompt Test

This test checks whether the GLM pipeline handles ambiguous or malformed input safely.

Test Case: Incomplete / Ambiguous Resume Input

- **Prompt Input:** A resume containing vague phrases such as "worked on many systems," missing job dates, and incomplete skill descriptions, with no clear role alignment.
- **Expected Behaviour:** The GLM should return a cautious structured output rather than overconfident conclusions. Missing or weak evidence should be reflected in the summary or confidence-related fields. The system should avoid promoting unsupported details into downstream workflow decisions.
- **Actual Behaviour:** The GLM produced a limited structured output with reduced specificity. Weak evidence in the resume did not result in fabricated skills or exaggerated experience claims. The workflow remained valid and could proceed with a lower-confidence evaluation or be routed for review.

Status: Pass

6.4 Hallucination Handling

Because HireFlow depends on GLM for structured hiring decisions, hallucination mitigation is built into the AI processing pipeline.

The system reduces hallucination risk through the following mechanisms:

1. **Schema-Constrained Output Validation**

   All GLM responses must conform to predefined JSON structures before they are accepted by downstream components.

2. **Field-Level Consistency Checks**

   Key fields such as skills, years of experience, recommendation, and decision type are checked for completeness, validity, and alignment with the source input and prior workflow state.

3. **Prompt Scoping**

   Prompts are narrowly scoped to task-specific outputs such as extraction, matching, or routing, instead of asking the model to make broad unbounded judgments.

4. **Fallback and Retry Logic**

   If the response is malformed, incomplete, or inconsistent, the system retries or routes the workflow into fallback handling instead of progressing automatically.

5. **Human Review for Low-Confidence Cases**

   Borderline or inconsistent outputs are routed to review rather than being treated as final decisions.

These controls ensure that GLM outputs are used as **validated workflow inputs**, not blindly trusted free-form responses.